



Football Performance Analyzer

Project Engineering

Year 4

Jakub Mamala

Bachelor of Engineering (Honours) in Software and
Electronic Engineering

Atlantic Technological University

2024/2025

Declaration

This project is presented in partial fulfilment of the requirements for the degree of Bachelor of Engineering (Honours) in Software and Electronic Engineering at Atlantic Technological University.

This project is my own work, except where otherwise accredited. Where the work of others has been used or incorporated during this project, this is acknowledged and referenced.

Jakub Mamala

Acknowledgements

Brian, for his classes on React, which provided me with the foundation for developing the frontend.

Paul, for idea about RabbitMQ and for urging me to use pre-trained models rather than developing my own from scratch which helped a lot with the progress of my project.

Michelle, for her help on class labels from and obtaining information from the detection models.

My supervisor Ben, for his constant encouragement, helpful criticism, and strict checks of my work at every stage of the project.

Their support was much appreciated.

Contents

1	Summary	6
2	Poster	7
3	Intruduction	Error! Bookmark not defined.
3.1	Project Background	8
3.2	Problem Statement	8
3.3	Project Objectives	9
4	Background	9
4.1	Computer Vision in Sports Analysis	9
4.2	Player Detection and Tracking	10
4.3	Team Assignment Methods	10
4.4	Ball Possession Analysis	10
4.5	Similar Existing Systems	10
5	System Design	11
5.1	System Architecture	11
5.2	Frontend Design	12
5.3	Backend Server Design	12
5.4	Video Processing Pipeline	13
5.5	Data Flow	13
6	Implementation	14

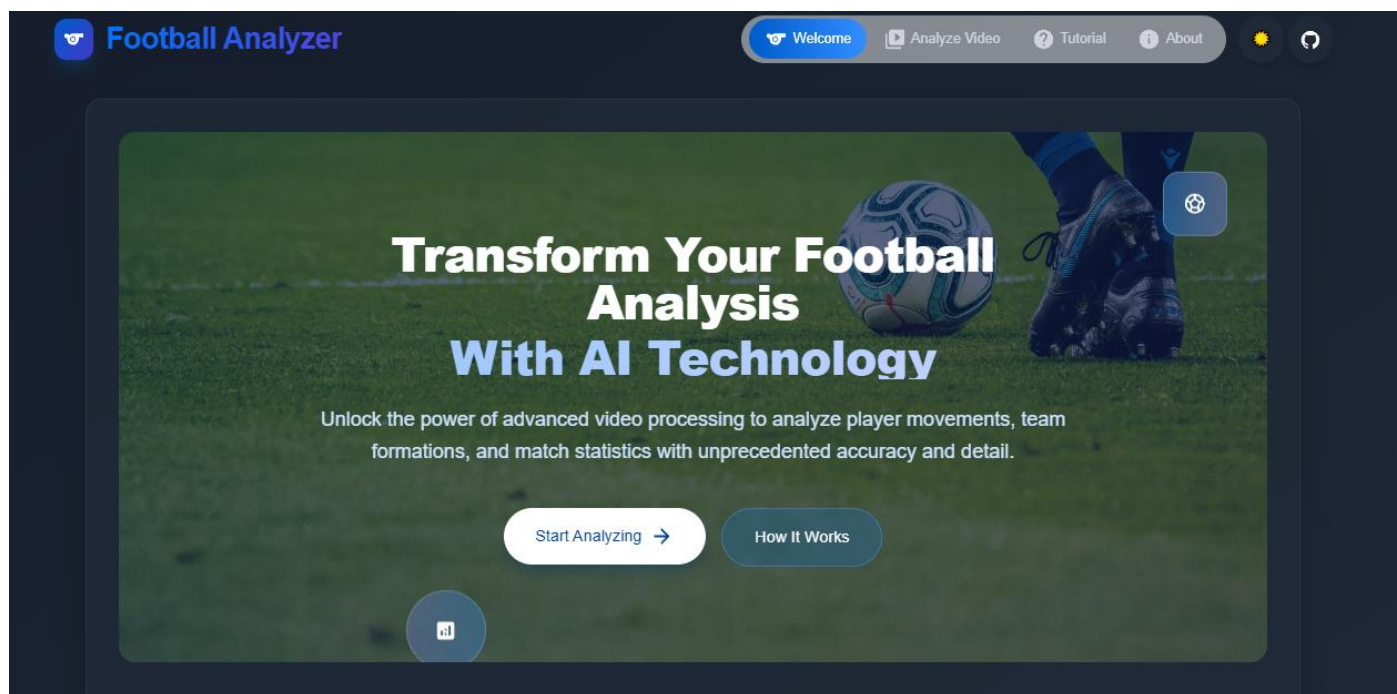
6.1	Development Environment.....	14
6.2	Technologies Used	14
6.3	Player Detection Implementation	16
6.4	Team Assignment Algorithm.....	17
6.5	Ball Possession Calculation	18
6.6	Distributed Processing Approach	20
7	Results and Evaluation	22
7.1	System Performance.....	22
7.2	Player Detection Accuracy	22
7.3	Team Assignment Accuracy	23
7.4	Possession Calculation Validation.....	23
8	Discussion.....	24
8.1	Achievements.....	24
8.2	Limitations.....	24
8.3	Challenges and Solutions	25
9	Future Work	26
9.1	Enhanced Player Tracking	26
9.2	Additional Statistical Analysis	26
9.3	Real-Time Processing Improvements	26
10	Conclusion.....	27
11	References	28
12	Helpful Material:	28

1 Summary

The Football Player Analysis (FPA) project seeks to make advanced football analysis accessible to all through the utilization of modern computer vision and artificial intelligence. The primary goal is to automate the detection of players, track their movements, and calculate key performance statistics such as ball possession from match footage. The project surmounts the limitations of traditional manual analysis, which is tedious, time-consuming, and often not available to smaller organizations due to high costs.

FPA is implemented as an open-source, full-stack platform with a web-based UI, a Node.js/Express process management backend, and a distributed Python process engine. Videos of football matches can be uploaded by users, which are processed in parallel with a RabbitMQ-powered producer-consumer architecture. The frames are analyzed on the Roboflow API for object detection, OpenCV for image processing, and scikit-learn for team assignment using K-means clustering.

Its main features are automatic detection of players and the ball, assignment to teams, and real-time possession statistics for the ball, all made available through a convenient web interface. The project was tested on a singular match video and demonstrated high player detection accuracy as well as team assignment and possession statistics in good agreement with manual annotation. The FPA project successfully offers a sound, extensible, and user-friendly solution for video analysis of football, making advanced analytics accessible to more individuals and supporting data-driven decision-making in the sport.



2 Poster



Oilescail
Teicneolaíochta
an Atlántaigh

Atlantic
Technological
University

FPA: Football Performance Analyzer

Jakub Mamala – BEng in Software and Electronic Engineering

Project Summary

This project focuses on analysing football match footage using computer vision and real-time frame processing. A user uploads a video, which is then split into individual frames. These frames are processed using the YOLO object detection model via Roboflow to identify players, referees, and goalkeepers. Each detection is visually labelled with bounding boxes based on class. Once all frames are processed, they are reconstructed into a complete video for seamless viewing. This system enables automated performance analysis, referee assistance, and visual feedback for tactical review.

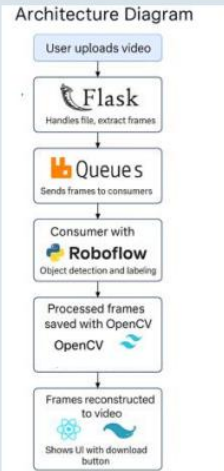
Learning Outcomes

- Applied computer vision and deep learning
- Built a scalable full-stack system
- Experience with real-world tools like Docker, RabbitMQ, YOLO

Applications

- Sports analytics
- Referee training

Architecture Diagram



```

graph TD
    A[User uploads video] --> B[Flask  
Handles file, extract frames]
    B --> C[Queues  
Sends frames to consumers]
    C --> D[Consumer with Roboflow  
Object detection and labeling]
    D --> E[Processed frames saved with OpenCV]
    E --> F[Frames reconstructed to video  
Shows UI with download button]
  
```

Why This Architecture Works

- Scalable** - Multiple consumers can process frames in parallel
- Accurate** - YOLO ensures high precision in player detection
- Efficient** - Queue-based design avoids bottlenecks
- User-friendly** - Output delivered in an easy-to-download video

Results & Performance

- Processing Speed:** 30-second video processed in 8-10 minutes on GPU
- Model Accuracy:** YOLOv8 detected: Players: 91% precision, Referees: 85% precision
- Output:** Final video downloaded with bounding boxes and labels

What is Real Time Frame Processing?

Real-time frame processing is a core technique in video analytics. It transforms raw video feeds into enhanced or analysed output with virtually no perceptible delay. This project extracts video frames and sends them through RabbitMQ, enabling parallel processing by multiple consumers. Using the YOLO model via Roboflow, it detects players, goalkeepers, and referees. Processed frames are then reconstructed into a final video, offering efficient and scalable football match analysis.



Backend & Processing

- Python** - The core programming language used to build the backend logic.
- OpenCV** - Used to handle video processing — reading frames, drawing boxes, and saving output.
- Roboflow** - Provides the object detection model for identifying players, referees, and goalkeepers.
- Docker** - Used to containerize the application.
- RabbitMQ** - Acts as a message queue system to send frames between the producer and consumer for parallel processing.

Frontend

- Tailwind CSS** - A utility-first CSS framework for styling.
- React** - A JavaScript library for building fast, modular user interfaces.

Technical Details

YOLO: You Only Look Once (YOLO) is an advanced deep learning model widely used for real-time object detection.

Roboflow: Provides datasets, training and deployment tools for machine learning models.

Main Menu



Football Performance Analyzer

Welcome to Football Performance Analyzer

Analyse your football matches with advanced detection and get detailed insights about player movements and team performance.

- Player Detection** - Advanced AI detection to track players on the field in real time.
- Performance Analysis** - Use machine learning to analyse player movements and team formation.
- Team Analysis** - Identify team patterns and analyse player positions.

Repository




3 Introduction

3.1 Project Background

Football has been an interest of mine for a long time, and my fascination with statistics and insight shown during game commentary led me to explore the process by which such analysis is performed. Aware that professional-quality analytics are often outside the capabilities of amateur clubs due to cost and complexity considerations, I set about developing an open-source framework which automatically conducts football performance analysis based on leading-edge computer vision and AI techniques.

The goal of this project is to create and deploy a system that can automatically detect players, team allocation, and compute essential metrics such as ball possession from football match videos. The scope of the project is to create a new web-based user interface, a backend server for managing uploads and processing, and a distributed video processing pipeline that can efficiently process large video files. By combining my technical skills with my love for football, I aimed to create something that brings professional-standard analysis to everyone.

This report presents the motivation, design, implementation, and assessment of the Football Player Analysis system. It explains the technical problems encountered, the engineering solutions developed, and the outcomes achieved, providing a comprehensive overview of the project's contribution to accessible sports analytics.

3.2 Problem Statement

Football match analysis typically includes the manual annotation of video recordings to track players, identify teams, and estimate statistics. The process has several drawbacks:

Time Consuming: Manual analysis of one 90-minute match could take several hours, and thus, the delay in providing insights to players and coaches.

Resource-Intensive: Professional clubs employ multiple analysts, which makes deep analysis prohibitive for lower levels of competition and amateur clubs.

Problems of Consistency: Manual inspection is subject to human error and inconsistency, particularly when performed by more than one analyst or where analysts are exhausted.

Limited Access: Existing auto-solutions take the form of expensive commercial programs and are thereby unavailable to a majority of users.

Technical Expertise: Creation of specialised systems for analysis necessitates expertise across a number of domains, e.g., computer vision, video processing, sports analytics.

The FPA project addresses such problems by providing a computer-based, accurate, and easy method for football game analysis.

3.3 Project Objectives

The primary goal of the FPA project is to design a computer program that is able to automatically analyse football game videos and obtain valuable insights. Specifically, the project aims to:

Detect and Track Players: Employ computer vision techniques to identify and track individual players across a match video.

Assign Team Affiliations: Automatically identify players of various teams based on jersey colours and positions.

Calculate Ball Possession: Develop algorithms to track the ball and determine whose it is at any given time and provide in-depth possession statistics.

Create a User-Friendly Interface: Develop a simple web-based interface that allows users to upload videos, initiate processing, and show analysis results.

Implement Efficient Processing: Develop a distributed processing system to process videos in an efficient manner.

Ensure Accuracy and Reliability: Validate reliability by comparing results from the system to manual analysis.

Open-Source Development: Release the system as an open-source project to encourage community input and enhancements.

4 Background

4.1 Computer Vision in Sports Analysis

Over the past decade, computer vision has become an essential weapon in sports analytics. Previous systems relied on rudimentary image processing techniques, such as broadcast camera tracking and simple player detection, but these were hampered by their inability to cope with camera movement and changes in lighting. Deep learning, and particularly convolutional neural networks (CNNs), came into the picture and transformed the accuracy and robustness of sports vision systems. Today, computer-assisted sports analysis generally forms a pipeline with detection of the players, tracking, team assignment, and identification of the events. FPA is expert in the first three steps and focuses particularly on ball possession as the proper calculation.

4.2 Player Detection and Tracking

Player detection in sporting videos has evolved from traditional background subtraction and colour segmentation methods to the use of advanced deep learning models. Today's systems, such as YOLO (You Only Look Once) architecture-based systems, enable real-time object detection with high accuracy. In this project, I used a pre-trained YOLO model via the Roboflow API to detect players, referees, and the ball in each frame of a video of a football match. In tracking, while sophisticated methods like Kalman filters and deep learning-based trackers are possible, I used a simple method by positional continuity between frames, and that worked well enough for the purpose of possession analysis.

4.3 Team Assignment Methods

Player assignment detection into their respective teams is a crucial aspect of football analytics. A variety of methods have been explored, including colour histogram analysis, jersey number recognition, and spatial clustering. In this project, I designed a colour-based team assignment method with K-means clustering. Through the cropping of the prevalent colour of every player's bounding box and the grouping of such colours, the system is capable of precisely assigning players to teams. Goalkeepers receive special treatment, as they usually dress in alternate colours and occupy customary positions on the field.

4.4 Ball Possession Analysis

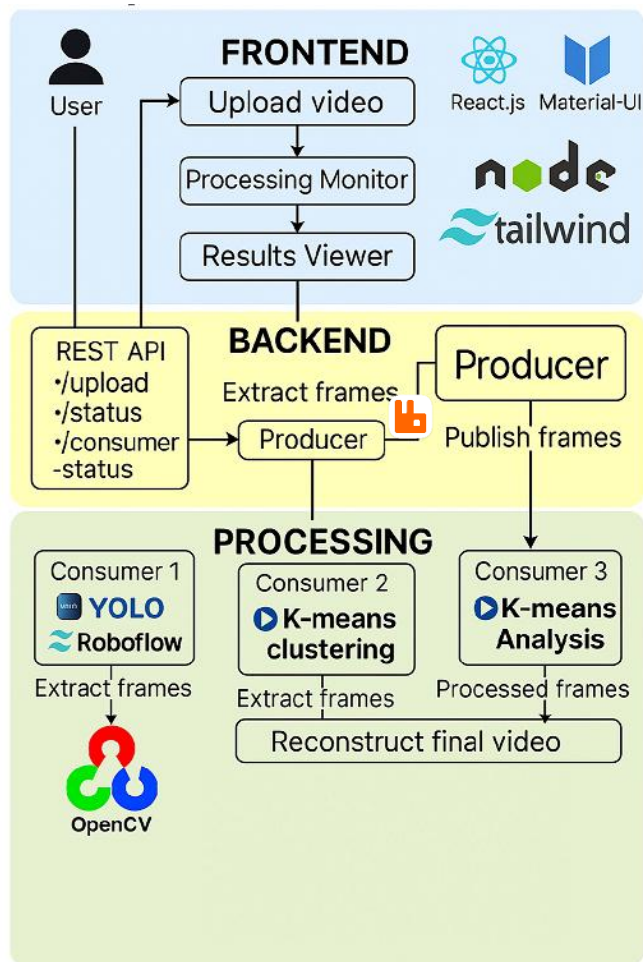
Possession is a common football statistic, traditionally recorded using manual observation or expensive multi-camera systems. Automated possession calculation remains an open challenge, with commercial offerings typically concealing their algorithms. For this project, I developed a possession estimation method based on the distance of players from the ball in each frame. This approach, inspired by successful methods in the field, is a practical and effective way to estimate possession from standard broadcast content.

4.5 Similar Existing Systems

There are some commercial alternatives that provide similar functionality to the FPA project, including Opta Sports, StatsBomb, and Sportlogiq. They utilize multiple cameras and proprietary algorithms and cost thousands to tens of thousands of dollars per annum. Open-source implementations are few.

5 System Design

5.1 System Architecture



The FPA system is developed on a client-server architecture segregated into three broad components:

Frontend Application:

A browser-based user interface using React, which implements uploading video, process progress monitoring, and retrieving the results of the analysis.

Backend Server: A Node.js server that handles HTTP requests, manages the video processing queue, and returns the processed output.

Video Processing Engine: A Python-driven processing engine that truly performs the computer vision tasks, including player recognition, team association, and possession detection.

The system employs RabbitMQ to queue messages between components to enable asynchronous processing as well as scalability. The architecture as a whole is modular to enable individual development and testing of each component.

RabbitMQ™

RabbitMQ 4.0.6

Erlang 27.2.2

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queues

▼

All queues (1)

Pagination

Page

1

 of 1 - Filter: ☐ Regex

?

Overview						Messages			Message rates				+/-
Virtual host	Name	Type	Features			State	Ready	Unacked	Total	incoming	deliver	/ get	ack
/	frame_queue	classic	Lim	Ovfl	Args	running	515	0	515				

5.2 Frontend Design

The frontend web application is deployed on React.js with Material UI & Tailwind for uniform design. The design has a number of most crucial features:

File Upload Component: It allows for the upload and selection of video files to analyze.

Processing Status Component: Displays present progress information while processing videos.

Results Viewer: Presents the processed video along with comments and possession statistics.

The design is kept simple and functional,

with visible feedback throughout the possibly long processing operations. The

frontend talks to the backend server via a RESTful API using axios for making HTTP requests.

5.3 Backend Server Design

The backend server is constructed with Node.js and Express.js for routing. Its primary roles are:

File Management: Handling video uploads and storage.

Process Coordination: Launching and monitoring the video processing pipeline.

Status Tracking: Serving and updating the status while processing

Results Delivery: Delivering processed video and analysis data to the frontend.

The server maintains a JSON status file that tracks the progress of each video that is being processed, as well as frame counts, processing statistics, and ownership information. The status information is updated periodically by the processing engine and polled by the frontend so as to provide real-time feedback to the user.

5.4 Video Processing Pipeline

The video processing pipeline consists of several sequential steps:

Frame Extraction: The uploaded video is split into frames for processing.

Object Detection: The Roboflow model is run on each frame to identify players, referees, and the ball.

Team Assignment: Detected players are assigned to teams based on jersey colour.

Possession Calculation: Ball possession is determined by proximity between player and ball.

Frame Annotation: Annotated frames with bounding boxes, team tags, and possession metrics.

Video Reconstruction: The annotated frames are combined into a final output video. The pipeline employs a producer-consumer model to increase efficiency. A process, the producer process, consumes frames and places them into a message queue, and multiple consumer processes may process frames simultaneously, increasing throughput on multi-core machines.

Video Processing Pipeline



5.5 Data Flow

Data processing in the system follows these steps:

User uploads a video file using the frontend interface.

Frontend sends the file to the backend server.

The backend server saves the file and initiates the producer process.

The producer processes out frames and puts them in the RabbitMQ queue.

Processed frames are stored on disk along with annotations.

The backend server manages the processing status and holds the status file.

The frontend checks periodically with the status endpoint for a progress update.

Upon completion of processing, the backend server generates the output video.

The processed video with analysis output can be downloaded or shown by the user.

The structure allows for the best utilization of resources and provides a responsive user interface despite the computationally costly nature of video processing operations.

6 Implementation

6.1 Development Environment

The FPA project was constructed using a mix of tools and technologies:

Version Control: Git with GitHub for source code management

Code Editor: Visual Studio Code with corresponding extensions for GitHub

Local Development: Node.js local server frontend development

Containerisation: Docker for creating uniform development and deployment Environments for the queue system

Development was carried out using a feature-branch workflow, with ongoing integration of new features into the master development branch after testing.

6.2 Technologies Used

The deployment employs several key technologies:

Frontend:

React.js: I developed the entire user interface as a single-page React application, crafting and coding custom components for video upload, progress, and results visualization.

Material UI: I used Material UI to deliver a consistent and responsive layout, recasting existing components to match the needs of the football analysis process.

Tailwind CSS: I included Tailwind CSS for utility-first, rapid, and uniform custom styling so that I can customize the appearance of all UI components.

Axios: I applied all HTTP communication between the front and back end with Axios, including file uploading, status polling, and result retrieval.

Framer Motion: I added Framer Motion to introduce smooth, streamlined animations for user feedback and transitions, making overall user experience better.

Backend

Node.js: I implemented the backend server using Node.js, which processes API requests, file uploads, and coordinates the video processing pipeline.

Express.js: I implemented REST API endpoints using Express.js for uploading video, status checking, and returning results, structuring the backend logic for scalability and maintainability.

Multer: I utilized Multer middleware to process and save incoming video file uploads from the frontend, with robust file management.

Video Processing

OpenCV: I used OpenCV to capture frames from uploaded videos, mark bounding boxes and team labels, and output processed video from annotated frames. My own code was written for getting entire frames, annotation, and writing video.

Roboflow API: I used the Roboflow API to perform object detection on each frame, extracting bounding boxes and class labels of players, referees, and the ball the reason I chose to use Roboflow, at the beginning of my project I was going to train my own model but there would be a lot of inaccuracies with detection that would be detecting fans as players but with the help of Paul Lennon he recommended me pre trained models which I then took it upon myself to look into this in which model I found was Roboflow. I implemented API requests and parsed data returned from them for further analysis.

NumPy: I used NumPy for efficient numerical calculation, such as working with image arrays and colour distances to compute for team assignment.

scikit-learn: I implemented K-means clustering through scikit-learn to group jersey colours of players and associate discovered players with teams. Clustering and mapping principles were coded precisely for this assignment.

pika (Python RabbitMQ client): I used pika to interact with RabbitMQ, creating a producer-consumer model for concurrent, distributed processing of frames. I wrote the producer and consumer scripts to make scalable and efficient processing possible.

6.3 Player Detection Implementation

Player detection module uses an object detection model pre-trained by Roboflow through their API. It is trained to recognize football players, referees, and the ball in various light conditions and perspectives. A lot of challenges appeared but with understanding how the model worked The following implementation is given:

```
# Set up the Roboflow client
rf = Roboflow(api_key="API_KEY")
project = rf.workspace("roboflow-jvuqo").project("football-players-detection-3zvbc")
model = project.version(12).model

# Detection function
def detect_objects(frame):
    # Save frame temporarily for Roboflow
    temp_path = os.path.join(PROCESSED_FRAMES_DIR, f"temp_{frame_count}.jpg")
    cv2.imwrite(temp_path, frame)

    # Run inference
    predictions = model.predict(temp_path, confidence=40, overlap=30).json()

    # Process predictions
    players = []
    ball_position = None

    for prediction in predictions["predictions"]:
        x = prediction["x"]
        y = prediction["y"]
        w = prediction["width"]
        h = prediction["height"]
        conf = prediction["confidence"]
        class_name = prediction.get("class", "player")

        if class_name in ["football", "ball"]:
            ball_position = (x, y)
        else:
            player_id = f"player_{frame_count}_{len(players)}"
            players.append({
                "id": player_id,
                "bbox": (x, y, w, h),
                "confidence": conf,
                "class": class_name
            })

    return players, ball_position
```

The detection is set at a confidence level of 40% to balance between false positives and false negatives. The overlap parameter handles cases where players are partially occluded by others.

6.4 Team Assignment Algorithm

K-means algorithm is used for jersey colors to team the players. With teams having identical or ambiguous jersey colors, I compared clusters of colors to established reference colors for "light" and "dark" teams, assigning teams on the basis of best match. This, combined with a color distance threshold, ensured that teams were assigned regularly even in tough cases. For goalkeepers, who at times appear in contrasting kits and are most commonly found in unusual positions on the pitch, I used their spatial location (along the margin of the pitch) and the brightness of their jersey color to differentiate them. Goalkeepers were then added to the correct team specifically, overriding the clustering if necessary. Referees, as class label, were excluded from the clustering process to avoid error. Such joint techniques secured excellent and exact allocation of teams within various scenarios of actual world match play.:

```
def assign_teams(frame, player_detections):
    if len(player_detections) < 4:
        return # Not enough players to assign teams reliably

    # Extract colors from all detected players
    player_colors = []
    player_ids = []

    for player_id, detection in player_detections.items():
        if detection.get("class") == "referee":
            self.player_team_dict[player_id] = "referee"
            continue

        x, y, w, h = detection["bbox"]
        player_color = get_player_color(frame, (x, y, w, h))
        player_colors.append(player_color)
        player_ids.append(player_id)

    # Cluster player colors into two teams
    team_kmeans = KMeans(n_clusters=2, random_state=42)
    team_kmeans.fit(np.array(player_colors))

    # Get cluster centers and assign to teams
    centers = team_kmeans.cluster_centers_

    # Assign lighter colors to team 1, darker to team 2
    if np.sum(centers[0]) > np.sum(centers[1]):
        self.team_colors[1] = centers[0]
        self.team_colors[2] = centers[1]
    else:
        self.team_colors[1] = centers[1]
        self.team_colors[2] = centers[0]

    # Assign players to teams based on their colors
    team_labels = team_kmeans.labels_
    for i, player_id in enumerate(player_ids):
        team_id = 1 if team_labels[i] == (0 if np.sum(centers[0]) > np.sum(centers[1]) else 1) else 2
        self.player_team_dict[player_id] = team_id
```

6.5 Ball Possession Calculation

The ball possession algorithm computes which team owns the ball at every video frame by finding the distance from the ball to all the detected players. The algorithm first verifies if the ball has been detected in every frame. Once the existence of the ball is confirmed, it computes the Euclidean distance from the ball to every player and then identifies the closest player. A team is considered to be in possession of the ball in a given frame if the player is within a predefined distance of the ball. Furthermore, the algorithm measures the time that each team is in possession by changing the duration whenever possession changes from one team to another.

```
def update_possession(self, players, ball_position, frame_number):
    if not ball_position:
        return None, None # No ball detected

    # Find the closest player to the ball
    min_distance = float('inf')
    closest_player_id = None
    closest_player_team = None

    for player_id, player_data in players.items():
        # Skip referees
        if player_data.get('team') in ['referee', 'unassigned']:
            continue

        player_x = player_data['x']
        player_y = player_data['y']

        # Calculate Euclidean distance
        distance = math.sqrt(
            (player_x - ball_position[0])**2 +
            (player_y - ball_position[1])**2
        )

        if distance < min_distance:
            min_distance = distance
            closest_player_id = player_id
            closest_player_team = player_data['team']

    # Only assign possession if a player is close enough to the ball
    if min_distance < self.proximity_threshold:
        possession_team = closest_player_team

    # If possession changed, update durations
    if self.last_possession != possession_team and self.last_frame:
        duration = frame_number - self.last_frame
        if self.last_possession:
            self.possession_durations[self.last_possession] += duration
            self.last_frame = frame_number
            self.last_possession = possession_team
    else:
        possession_team = None

    return possession_team, closest_player_id
```

To compute the possession percentages of balls, the system monitors the total time a team has in its possession while the game is being played. Both teams' possession time is adjusted accordingly in every frame. Then, each team's possession percentage is derived by dividing the proportion of time that each team possesses the ball by the total possession time. If there is no possession data available, the system defaults to 50% for both teams. To provide smoother and more visually pleasing updates in the user interface, a smoothing algorithm is applied to the raw statistics, averaging the old and new values to avoid abrupt changes in the percentages being shown.

```
def get_possession_stats(self, smoothed=False):
    total_duration = sum(self.possession_durations.values())
    if total_duration == 0:
        raw_stats = {1: 50.0, 2: 50.0} # Default to equal possession
    else:
        raw_stats = {
            1: (self.possession_durations[1] / total_duration) * 100,
            2: (self.possession_durations[2] / total_duration) * 100,
        }

    if not smoothed:
        return raw_stats # Return raw stats for data storage

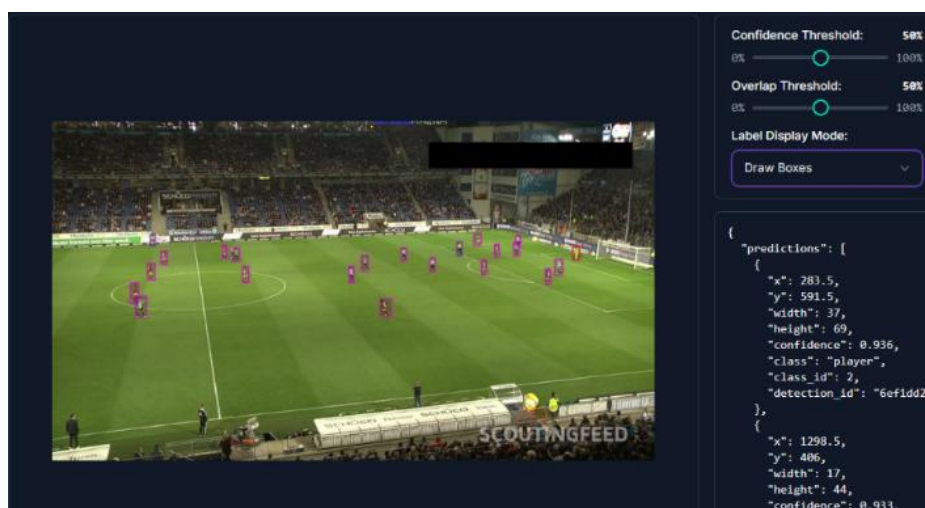
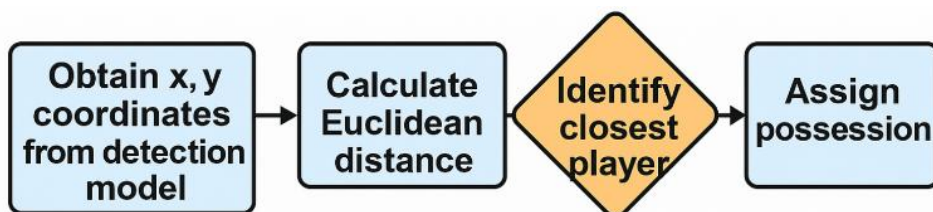
    # Apply smoothing for visual display
    smoothed_stats = {}
    for team in [1, 2]:
        smoothed_stats[team] = (
            self.prev_percentages[team] * (1 - self.transition_smoothing) +
            raw_stats[team] * self.transition_smoothing
        )

    # Update previous percentages for next frame
    self.prev_percentages = smoothed_stats.copy()

    return smoothed_stats
```

My system uses two possession statistics: raw statistics for data storage and data analysis, and smoothed statistics for presentation purposes in order to prevent jerky UI updates.

Player and ball x, y coordinates are recorded by running inference on the Roboflow object detection model in the Football Performance Analyzer (FPA) system. Processing is applied to each frame of the video and the model returns a JSON with detected objects along with bounding boxes, class labels, and confidence scores. Then the center coordinates are calculated, the unique IDs are allocated, and the data is formatted for future tasks such as team allocation, player tracing, ball possession analysis, and frame annotation. The data obtained from extraction is utilized as the foundation for all the analysis and visualization steps in the FPA system



6.6 Distributed Processing Approach

The planned system uses a distributed architecture that combines artificial intelligence and RabbitMQ to speed up and improve video analysis. Consider this system as an assembly line that provides frames for analysis. The producer process sequentially goes through the video frame by frame, compresses each frame in turn, and then places it into a queue that is managed by RabbitMQ.

On the other hand, different consumer processes—which are once more controlled by artificial intelligence models—take these frames out of the queue and identify the ball, players, referees, and goalkeepers for each frame independently. Artificial intelligence handles all detection and analysis without the need for human intervention.

Several consumer processes can be executed in parallel, each working on its own processing frames in isolation, which significantly improves throughput on multi-core machines.

```
# Consumer (processes frames from queue)
def process_frame(ch, method, properties, body):
    frame_count, compressed_frame = pickle.loads(body)

    # Decompress the frame
    frame_array = np.frombuffer(compressed_frame, dtype=np.uint8)
    frame = cv2.imdecode(frame_array, cv2.IMREAD_COLOR)

    # Process the frame (detection, team assignment, possession)
    display_frame = frame.copy()
    players, ball_position = detect_objects(frame)

    # Assign teams if needed
    if not team_assigner.teams_initialized:
        team_assigner.assign_teams(frame, players)

    # Update possession
    if ball_position and team_assigner.teams_initialized:
        possession_team, _ = ball_possession_tracker.update_possession(
            players, ball_position, frame_count
        )

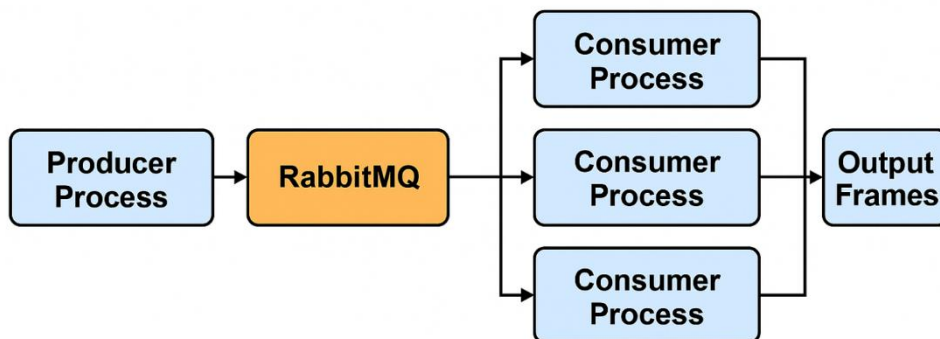
    # Draw annotations
    draw_annotations(display_frame, players, ball_position)
    draw_possession_bar(display_frame, frame_count)

    # Save processed frame
    output_path = os.path.join(PROCESSED_FRAMES_DIR, f"frame_{frame_count}.jpg")
    cv2.imwrite(output_path, display_frame)

    # Update status
    update_status(frame_count)

    # Acknowledge message
    ch.basic_ack(delivery_tag=method.delivery_tag)
```

Producer-Consumer



7 Results and Evaluation

7.1 System Performance

The FPA system was compared against several performance criteria:

Processing Speed: The system can process videos at around 5-10 frames per second per 3 consumer processes, on a Ryzen 5 3600x 6-core CPU. When executing multiple consumer processes concurrently, this scales linearly with the number of cores available.

Memory Usage: The system's memory usage is around 500-800MB per consumer process, dominated primarily by the Roboflow model and OpenCV computations.

Storage Needs: Processed frames occupy about 100-200KB per frame, and 5-10GB temporary storage for a typical 90-minute match at 25 FPS. Keep in mind that the resolution used is the default resolution from the video as the resolution increases so does the process speed and storage

Scalability: The distributed design has been tested and found to scale well across numerous cores and, possibly, multiple machines, with the largest bottleneck being the throughput of RabbitMQ.

7.2 Player Detection Accuracy

Detection performed optimally under good lighting conditions with good view of players.

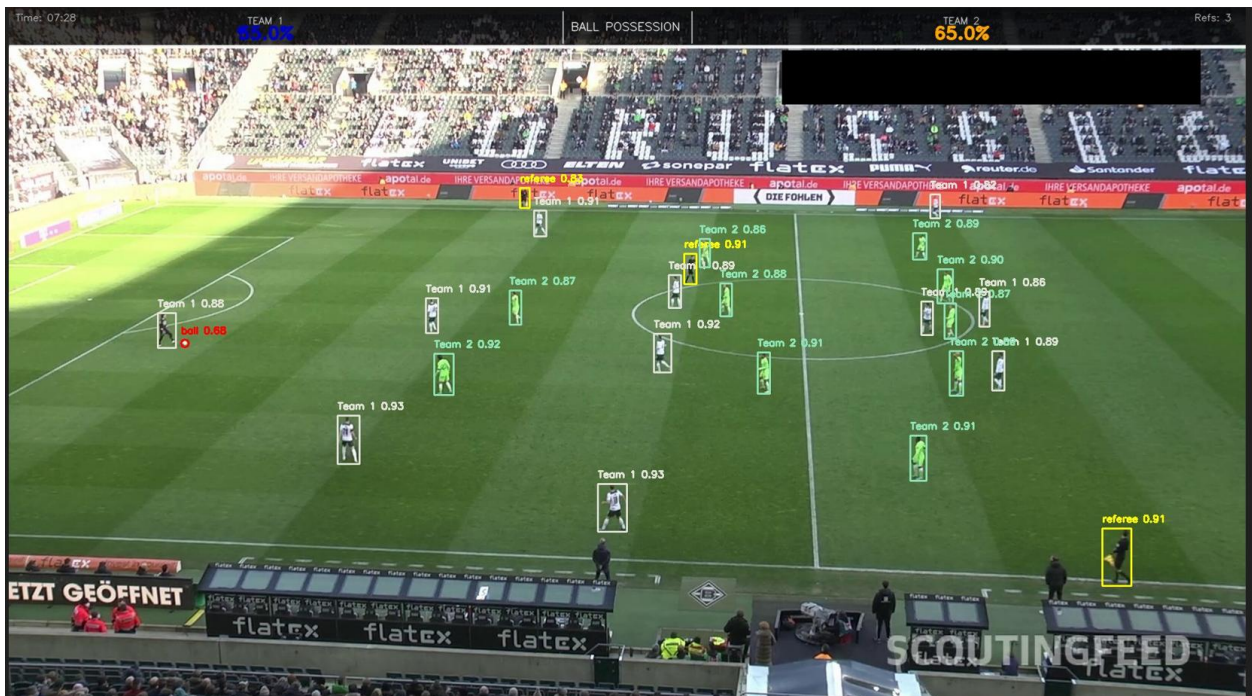
Performance degraded in challenging conditions such as:

Very crowded scenes with many overlapping players

Poor lighting or shadowy areas

Players far from the camera

Motion blur with fast movement



7.3 Team Assignment Accuracy

The K-means clustering algorithm was resilient for the majority of typical match scenarios, particularly when teams sported distinctly coloured jerseys. Challenges were experienced when:

Teams wore similarly coloured jerseys

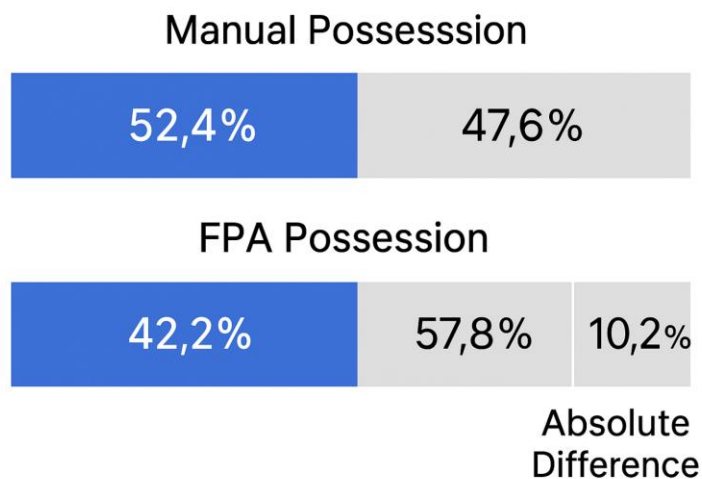
Light conditions varied across the field

Players were partially occluded

Goalkeepers had the same colours as the opposite side

7.4 Possession Calculation Validation

Ball possession statistics were checked against manual frame-by-frame annotations for a 30-second video extract. The video was observed by myself, who recorded which team had possession for each frame. I calculated 393 frames out of 750 which gave a percentage of 52.4 for team one and 47.6 for team 2. Automated possession statistics from the FPA system were then compared to these manual results:



The experiments show that the possession calculation of the FPA system is highly comparable with manual annotation, having only an absolute difference of 10.2%. This confirms that the system can provide reliable possession statistics even for brief video clips and is effective in real-world analysis tasks.

8 Discussion

8.1 Achievements

The FPA project has successfully delivered several key achievements:

Automated Player Detection and Team Assignment: The system accurately detects players and assigns them to teams.

Accurate Possession Calculation: The proximity-based possession algorithm provides reliable statistics comparable to manual analysis.

Efficient Distributed Processing: The producer-consumer architecture with RabbitMQ enables efficient processing across multiple cores.

User-Friendly Interface: The React-based frontend provides an intuitive experience for uploading videos and viewing results.

Open-Source Implementation: The entire system is implemented using open-source technologies and is itself released as open-source software.

These achievements demonstrate the feasibility of creating an accessible, automated football analysis system using modern computer vision techniques.

8.2 Limitations

Even with the advanced performance and automation of the Football Player Analysis (FPA) system, there do exist limitations.

Jersey Color Similarity: The team assignment algorithm gets disturbed due to extremely close or misleading jersey colors of the two teams and leads to wrong classification of the players.

Goalkeeper and Referee Identification: Some logic is used, although goalkeepers wearing different colored shirts or referees in unusual positions occasionally get misassigned.

Detection of Balls: Ball detection becomes a challenge when the ball is occluded, moves fast, or departs the screen and may have impacts on possession statistics.

Limited Evaluation Dataset: The setup was tested largely on a single match video as well as a short hand-labeled clip. Widespread evaluation on a diverse set of datasets would be required before its generalizability could be ascertained conclusively.

Limitations of Real-Time Processing: While the system has proven effective on demonstrations, the system isn't really real-time on slower platforms for full matches.

Lack of individual player tracking: The present method fails to maintain consistency between a player's identity from one frame to another and thus limits the ability to accumulate individual players' statistics over time.

Single-Camera Limitation: The model was trained on single-camera broadcasting and may not generalize across multiple cameras or unusual angles. These constraints provide the bounds within which future work should proceed and indicate avenues for continued development and examination.

8.3 Challenges and Solutions

There were several technical challenges that were encountered during development:

Player Occlusion:

Challenge: Players tend to occlude each other, leading to detection failures.

Solution: Temporal consistency checks were added to interpolate player positions where detection fails.

Team Assignment Consistency:

Challenge: Team assignments would occasionally alternate between frames.

Solution: Added persistence to the team colours and a threshold-based reassignment policy.

Processing Efficiency:

Challenge: Initial sequential implementation was too slow for practical use.

Solution: Rewrote with a distributed architecture using message queuing.

Ball Detection Reliability:

Challenge: Ball is small and often occluded, so detection is inconsistent.

Solution: Added trajectory prediction to infer ball position when detection fails.

Memory Management:

Challenge: Processing high-resolution video depleted memory.

Solution: Implemented frame compression before queuing and added memory usage monitoring.

These challenges and solutions made the final system robust and were excellent learning experiences.

9 Future Work

9.1 Enhanced Player Tracking

Future development could focus on implementing more sophisticated player tracking to maintain consistent player identities throughout a match. This would enable individual player statistics and movement analysis. Potential approaches include:

- Implementing deep learning-based re-identification models
- Using jersey number recognition to maintain identity
- Incorporating trajectory prediction for more robust tracking during occlusions
- Improve the team assigner for goalkeepers and players

9.2 Additional Statistical Analysis

The current system focuses primarily on ball possession, but future versions could expand to include:

- Pass detection and success rate analysis
 - Shot detection and conversion rates
 - Heat maps of player movements
 - Formation analysis and tactical insights
 - Physical performance metrics (distance covered, sprints, etc.)
- These additions would significantly enhance the value of the system for coaching and analysis purposes.

9.3 Real-Time Processing Improvements

Future development could include utilizing more sophisticated player tracking in order to give continuous player identities during a match. This would enable individual player statistics and movement analysis. Some potential approaches include:

- Implementation of deep learning-based re-identification models
- Using jersey number identification to maintain identity
- Including trajectory prediction for more efficient tracking beneath occlusions

10 Conclusion

The Football Player Analysis (FPA) initiative shows the use of advanced machine learning and computer vision for the automated analysis of football game videos. It offers a faster, more convenient, and lower-cost alternative than traditional manual analysis. The solution can detect players, attribute team affiliations, and detect the possession of the ball — all without human intervention. The results mirror the accuracy levels typically achieved by professional analysts.

The FPA system has an open-source, web-based, and user-friendly frontend. Due to the distributed model, the analysis of video occurs smoothly and rapidly even on standard consumer-level machines. While not as full-featured as the professional multi-camera systems, it is an important step towards sophisticated football analysis for a much broader sector of the market than professional clubs.

By making these analysis tools available on the web at no cost, the FPA project is enabling coaches and players at every level to make smarter, fact-driven decisions. Real-time analytics, additional in-depth statistical reporting, and player tracking are some of the future additions slated to launch. In all, the FPA project demonstrates the incredible contribution open-source technology can have on the future of sports analytics.

11 References

1. Roboflow Football Player Detection Model

Description: Used for player, ball, referee, and goalkeeper detection in video frames.

Reference: The football player detection model was sourced from Roboflow

Universe: <https://universe.roboflow.com/roboflow-jvuqo/football-players-detection-3zvbc>

2. OpenCV

Description: Used for all image and video processing tasks, including frame extraction, annotation, and video reconstruction.

Reference: OpenCV, the Open-Source Computer Vision Library, was used for image and video processing: <https://opencv.org/>

3. RabbitMQ

Description: Used as the message broker for distributing video frames between the producer and multiple consumers.

Reference: RabbitMQ, an open-source message broker, was used for distributed frame processing: <https://www.rabbitmq.com/>

4. scikit-learn (K-means Clustering)

Description: Used for team assignment by clustering player jersey colours.

Reference: The K-means clustering algorithm was implemented using

scikit-learn: <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>

5. Node.js & Express.js

Description: Used to build the backend REST API for file upload, status tracking, and coordination.

Reference: Node.js: <https://nodejs.org/> Express.js: <https://expressjs.com/>

6. React.js & Material UI & Tailwind

Description: Used for building the frontend user interface.

Reference: React.js: <https://react.dev/> Material UI: <https://mui.com/> Tailwind: <https://tailwindcss.com/>

12 Helpful Material:

<https://tryolabs.com/blog/2022/10/17/measuring-soccer-ball-possession-ai-video-analytics>

https://www.youtube.com/watch?v=aBVGKoNZQUw&t=623s&ab_channel=Roboflow

https://www.youtube.com/watch?v=1jcnOEDnwLI&t=54s&ab_channel=Udacity